How to Design a Database for Online Banking System

Last Updated : 6 Aug, 2025

Designing a **relational database** for an **online banking system** is an important and interesting task that requires careful **planning** and consideration. The database serves as the backbone of the **banking system** that **stores** and **organizes** large amounts of financial data **securely**.

In this guide, we will explore **How to Design a Relational Database for an Online Banking System** by designing the **entity relationship** diagram and **Database Model** with the help of various required **entities**, **attributes,** and **relationships**.

Database Design for Online Banking System

The database for the **Online Banking System** must efficiently manage **Account Details**, **Customer Details,** and **Transactions**. **Users** should be able to log in to the system and perform banking operations like **balance check**, **deposit**, **withdraw** and **money transfer**. Let's understand the overview of the **Online Banking System** through the below terms.

1. **User Management:**
   - We should manage user accounts which include registration, login and authentication.
   - Store user details such as name, email, address, and contact information securely.
2. **Account Management:**
   - Maintain records of bank accounts associated with each user.
   - Track account balances, transaction history, and account status (active, closed, etc.).
3. **Transaction Processing:**
   - Handle various types of transactions, including deposits, withdrawals, transfers, and bill payments.
   - Ensure transactions are secure, accurate, and recorded in real time.
4. **Security Features:**
   - Implement robust security measures to protect sensitive data and prevent unauthorized access.
   - Use encryption techniques to secure data transmission over the network.
5. **Reporting and Analytics:**
   - Generate reports on account activity, transaction history, and user behavior.
   - Use analytics to identify trends, detect fraud, and improve service offerings.
6. **Customer Support:**
   - Provide customer support features, such as chat support, FAQs, and helpdesk services.
   - Maintain records of customer interactions and feedback for improving services.

Online Banking System Features

- **User Login**: The user login feature is a important component of online banking systems by providing a secure gateway for users to access their accounts. Users must authenticate themselves with a valid **user ID** and **password** and sometimes additional security measures like OTP or biometric authentication are used for added security.
- **Check Balance:** After logging in the users can check the available balance in their bank accounts. This feature provides users with **real-time** information about their finances and helping them to manage their funds more effectively.
- **Send Money:** Online banking systems allow users to transfer money from their account to another valid account. Users need to provide the **recipients account number** and other necessary details to complete the transaction securely.
- **Add Beneficiary:** To make fund transfers easier and faster users can add beneficiaries to their account. Adding a beneficiary requires providing the recipient's account details and verifying the **relationship**, ensuring that the transfer is **authorized**.
- **Receive Money:** When a user receives money into their account, the transaction is recorded and the account balance is updated accordingly. This feature allows users to receive funds from other users or **external** sources.
- **Transaction History**: Users can view their transaction history, which includes details such as the **amount**, **time** and type of **transaction** like **deposit**, **withdrawal**, fund transfer. This feature helps users keep track of their financial activities and monitor their spending patterns.

Entities and Attributes of Online Banking System

1. Customer

- **CustomerID (primary key):** Unique identifier for each customer
- **Name:** Name of the customer.
- **Address:** Address of the customer.
- **Contact**: Contact details of the customer (e.g.: Phone number).
- **Username (Unique) :** username to login to the banking system.
- **Password:** Encrypted password of the user.

2. Account

- **AccountID (primary key):** Unique identifier of account.
- **CustomerID (foreign key referencing Customer):** Identifier of the customer who owns the account.
- **Type:** Defines the account type whether savings, current etc.
- **Balance:** Amount of money available in the account.

3. Transaction

- **TransactionID (primary key):** Unique identifier of the transaction, automatically increments.
- **AccountID (foreign key referencing Account):** Identifies the account where the transaction took place.
- **Type**: Defines the transaction type i.e. deposit or withdrawal.
- **Amount**: Shows the balance amount used for the transaction.
- **Timestamp:** The date and time of the transaction.

4. Beneficiary

- **BeneficiaryID (primary key):** Unique identifier of the beneficiary.
- **CustomerID(foreign key referencing Customer):** Identifies the customer who added the beneficiary.
- **Name:** Name of the beneficiary.
- **AccountNumber:** Account Number of the beneficiary.
- **BankDetails:** Bank Name of the Beneficiary added.

Relationships Between These Entities

Customer - Account relationship

- **One to Many Relationship:** One customer is allowed to create many accounts.
- **Foreign Key:** CustomerID in Account Table is referencing CustomerID in Customer Table.

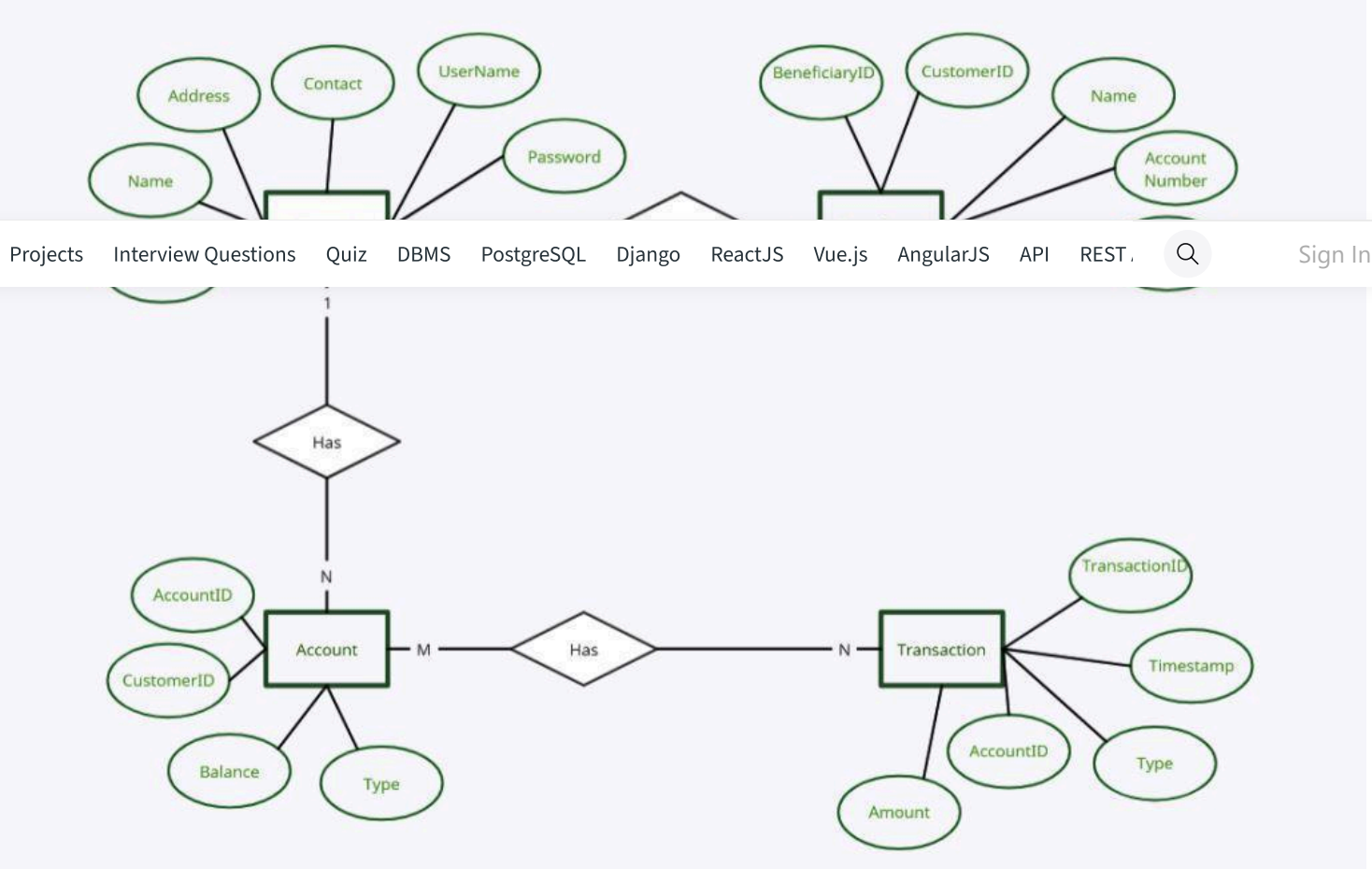
Customer - Beneficiary Relationship

- **One to Many Relationship:** A customer can add multiple beneficiaries.
- **Foreign Key:** CustomerID in Beneficiary is referencing to CustomerID in Customer Table.

Account - Transaction Relationship

- **Many to Many Relationship:** An account can have multiple transactions and a transaction can involve multiple account numbers.
- **Foreign Key:** AccountID in Transaction is referencing to AccountID in Account Table.

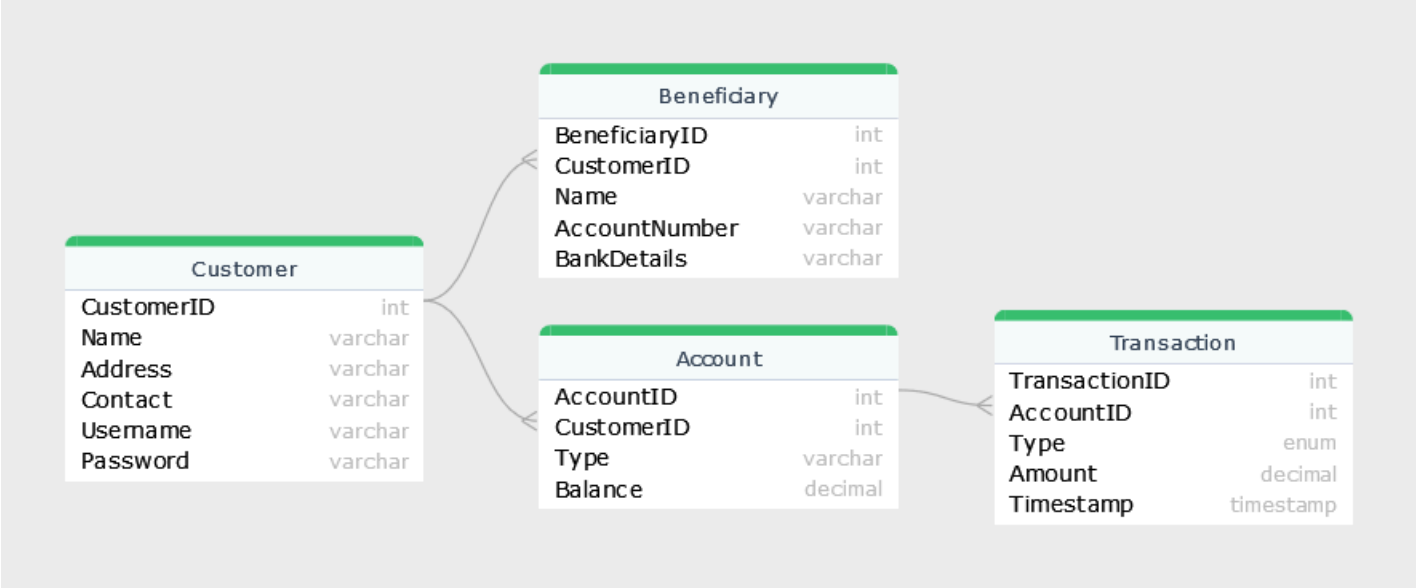
Representation of ER Diagram



L Projects Interview Questions Quiz DBMS PostgreSQL Django ReactJS Vue.js AngularJS API REST Sign In

ER Diagram of Online Banking System

Representation of Database Model



Database Model for Online Banking System

Entities Structures in SQL format

```
-- Create Customer table CREATE TABLE Customer
(     CustomerID INT PRIMARY KEY,
Name VARCHAR(255),
Address VARCHAR(255),
Contact VARCHAR(255),
Username VARCHAR(50) UNIQUE,
Password VARCHAR(255)
);

-- Create Account table CREATE TABLE Account (
AccountID INT PRIMARY KEY ,
CustomerID INT,     Type VARCHAR(50),
Balance DECIMAL(10, 2),
FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
);

-- Create Transaction table CREATE TABLE Transaction
(
TransactionID INT PRIMARY KEY AUTO_INCREMENT,
AccountID INT,
Type ENUM('deposit', 'withdrawal'),
Amount DECIMAL(10, 2),
 Timestamp TIMESTAMP,
FOREIGN KEY (AccountID) REFERENCES Account(AccountID) );

-- Create Beneficiary table CREATE TABLE Beneficiary
(
BeneficiaryID INT PRIMARY KEY ,
CustomerID INT,     Name VARCHAR(255),
AccountNumber VARCHAR(50),
BankDetails VARCHAR(255),
FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
);
```

Tips and Tricks to Improve Database Design

- **Normalize Your Database:** Normalize your database to reduce redundancy and dependency, enhancing data integrity and storage efficiency.

- **Use Indexes Wisely:** Use indexes on columns frequently used in search conditions or join operations to improve query performance.
- **Optimize Queries:** Write efficient SQL queries by minimizing joins, using WHERE clauses effectively, and limiting retrieved columns to improve query performance.
- **Implement Security Measures:** Apply role-based access control, encryption, and parameterized queries to prevent unauthorized access and protect your data.
- **Document Your Database Design:** Documenting your schema helps in understanding, maintaining, and collaborating with others, ensuring your database remains well-structured and scalable.

Conclusion

Designing a relational database for Online Banking System involves defining entities, attributes, and relationships for efficient store and management of data. A well designed database can help in providing valuable business insights and improves user experience. By following database design principles such as normalization and establishing proper relationships can create a scalable and effective database.

Comment

S subha... 2

Article Tags: SQL Dev Scripter 2024 Database Design

Explore

Basics

Queries & Operations

SQL Joins & Functions

Data Constraints & Aggregate Functions

Advanced SQL Topics

Database Design & Security

GeeksforGeeks
Sanchhaya Education Private Limited

Corporate & Communications Address:
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

Company
About Us
Legal
Privacy Policy
Contact Us
Advertise with us
GFG Corporate Solution
Training Program

Explore
POTD
Job-A-Thon
Blogs
Nation
Skill Up

Tutorials
Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core Subjects
Interview Preparation
Software and Tools

Courses
ML and Data Science
DSA and Placements
Web Development
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Videos
DSA
Python
Java
C++
Web Development
Data Science
CS Subjects

Preparation Corner
Interview Corner
Aptitude
Puzzles
GfG 160
System Design

GET IT ON Google Play
Download on the App Store

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved